

AI Internship Sprint

Day 2 Study Guide — Data Science, ML & Computer Vision

This guide covers everything built and learned on Day 2: the AI Prompt/Tools API (Project 3), the full ML overview notebook (ETL, EDA, classification, K-Means, PCA), and the Computer Vision app (OpenCV + DeepFace). Each topic is explained in plain language with a real example from the work done today, followed by a bank of likely interview questions with model answers.

What was built today	Status
Project 3 — AI Prompt/Tools API (chat + tool-call logging)	Done
ML Overview Notebook (ETL, 5 classifiers, K-Means, PCA)	Done
Computer Vision App (OpenCV + DeepFace, run on Colab)	Done

1. AI Prompt/Tools API (Project 3)

Separating Conversation Logs from Tool-Call Logs

Definition:

Two distinct database tables, PromptLog and ToolCall, instead of one table holding everything. PromptLog stores prompt/response pairs; ToolCall stores tool name, input, and output.

Where it's used:

Any system that mixes chat-style interactions with discrete tool/function calls -- which is exactly the shape of modern LLM agent systems.

Example:

POST /chat logs to PromptLog. POST /tools/calculate and POST /tools/summarize both log to ToolCall, tagged with a tool_name so calls can be filtered by tool.

Why it matters for this internship:

Mirrors how real agent frameworks separate conversation history from tool-use history -- directly relevant to the ReAct agent built on Day 3.

Rule-Based Fallback Pattern

Definition:

Building an endpoint's full architecture (logging, request/response shape, database schema) around a placeholder rule-based response, with the explicit intent of swapping in a real LLM call later without changing anything else.

Example:

/chat currently returns a templated string instead of calling a real LLM API, but still logs every prompt and response exactly as it would with a real model.

Why it matters for this internship:

Demonstrates understanding of the actual architecture an LLM-backed endpoint needs, independent of whether a live API key/model is available -- a strong, honest answer when asked about real LLM integration.

Input Sanitization with eval()

Definition:

Python's eval() runs arbitrary code from a string -- a known security risk if used on untrusted input. A simple defense is whitelisting which characters are allowed before evaluating.

Example:

/tools/calculate checks that the input expression only contains digits, operators, parentheses, and spaces before calling eval() on it.

Key terms:

Whitelisting (only allow known-safe characters) is generally safer than blacklisting (trying to block known-bad patterns), since blacklists are easy to miss edge cases for.

2. ETL & Data Cleaning (ML Notebook, Part 1)

Inspecting Before Cleaning (df.info())

Definition:

Checking every column's data type and how many non-missing values it has, before deciding how to clean anything.

Where it's used:

The mandatory first step of any real-world data science task -- you cannot make good cleaning decisions blindly.

Example:

df.info() on the Titanic dataset revealed age was missing for 177 of 891 passengers, and deck was missing for nearly 77% of rows.

Handling Missing Values

Definition:

Deciding, column by column, whether to drop a column (too sparse to be useful), fill missing values with a sensible estimate, or leave it.

Example:

age (177 missing) was filled with the median age. embarked (2 missing) was filled with the most common port. deck (~77% missing) was dropped entirely.

Key terms:

Median fill (numeric columns) -- robust to outliers, unlike the mean. Mode fill (categorical columns) -- fills with the most frequent category, since you cannot average text.

Encoding Categorical Variables

Definition:

Converting text categories into numbers, since ML models can only process numeric input.

Example:

sex was mapped to 0/1 directly. embarked (3 ports: C, Q, S) was one-hot encoded into embarked_Q and embarked_S, with C implied when both are 0.

Key terms:

Simple mapping (0/1) -- correct for exactly 2 categories with no inherent order, e.g. sex. One-hot encoding -- correct for 3+ categories with no order, creates a separate 0/1 column per category to avoid implying a false ranking.

Why it matters for this internship:

Choosing the right encoding technique per column type is a common practical interview question -- mapping all categories to 0/1/2 blindly would have wrongly implied an order between ports.

3. EDA & Classification Models (ML Notebook, Part 2)

Exploratory Data Analysis (EDA)

Definition:

Visualizing the data before modeling, to spot patterns and sanity-check assumptions.

Example:

A bar chart of survival rate by passenger class showed a clear pattern (63% for 1st class vs 24% for 3rd class), confirming pclass would likely be a useful predictive feature before any model was trained.

Why it matters for this internship:

Catches issues early -- a histogram of age also revealed an artificial spike at the median, caused by our own missing-value fill strategy, not a real pattern in the data.

Comparing Multiple Classification Algorithms

Definition:

Training several different algorithms on identical training data, then testing all of them on identical held-out test data, to choose the best performer for a real reason.

Example:

On the Titanic dataset: Random Forest scored highest (81%), followed by Logistic Regression (80%), Decision Tree (79%), Naive Bayes (78%), and KNN lowest (70%) -- KNN underperformed because features were never scaled, and it is a distance-based algorithm sensitive to differing feature ranges (e.g. fare vs. sex).

Key terms:

Logistic Regression -- weighted combination of features. KNN -- predicts based on the most similar nearby data points; sensitive to feature scale. Decision Tree -- a series of learned yes/no splits. Random Forest -- many decision trees voting together, generally more robust than one tree. Naive Bayes -- probability-based, assumes features are independent.

Why Random Forest Outperforms a Single Decision Tree

Definition:

A single tree can be unstable and prone to overfitting, since it learns one specific set of splits from the training data. Random Forest trains many trees on random subsets of data/features, then has them vote -- individual trees' errors tend to be different and cancel out when combined.

Example:

Analogy: one person's guess can be far off, but the average of many independent guesses tends to land closer to the truth.

4. Unsupervised Learning: K-Means & PCA (ML Notebook, Part 3)

K-Means Clustering

Definition:

An unsupervised algorithm that groups similar data points together into k clusters, using only feature values -- no labels involved at all.

Example:

Clustering Titanic passengers by class/age/fare into 3 groups (with k=3) produced clearly interpretable clusters: a low-fare/3rd-class majority, a wealthy all-1st-class elite group, and a mid-tier wealthier group -- without ever using the survived label.

Key terms:

The algorithm places k center points, assigns each data point to its nearest center, then repeatedly moves each center to the average position of its assigned points, until the groups stabilize.

Why it matters for this internship:

Demonstrates the core supervised vs. unsupervised distinction: classification predicts a known label using labeled data; clustering finds unknown, natural groupings with no labels at all.

PCA (Principal Component Analysis)

Definition:

A dimensionality reduction technique that compresses many features into fewer 'combined' axes (components), preserving as much of the original variation as possible -- commonly used to make high-dimensional data visualizable in 2D.

Example:

5 Titanic features compressed to 2 components preserved about 67% of the original variation; plotting the result and coloring by survival showed survivors and non-survivors were not cleanly separated in just 2D -- consistent with the ~81% (not near-100%) classification accuracy seen earlier.

Key terms:

`explained_variance_ratio_` reports how much of the original information each component preserved. Features must be scaled first (e.g. with `StandardScaler`) since PCA, like KNN, is sensitive to differing feature ranges.

Other Clustering & Dimensionality Reduction Methods (Notes Only)

Definition:

Conceptual awareness of related techniques not directly implemented this sprint.

Key terms:

Hierarchical Clustering -- builds a tree of clusters by repeatedly merging the closest pairs. DBSCAN -- groups by density, automatically finds outliers, no need to choose k upfront. Gaussian Mixture Models (GMM) -- like K-Means but 'soft', giving probabilities of belonging to each cluster. t-SNE -- similar goal to PCA but better for preserving local structure in non-linear data. Autoencoders -- a neural network that compresses then reconstructs data, acting as a learned, non-linear alternative to PCA.

5. Computer Vision: OpenCV + DeepFace

OpenCV Face Detection (Classical Computer Vision)

Definition:

A rule-based, pattern-matching approach to finding faces in an image -- answers only 'is there a face here, and where', with no understanding of who or what expression.

Example:

detectMultiScale() scanned the test image at multiple scales and returned one bounding box, correctly drawn around the detected face.

Key terms:

Haar Cascade Classifier -- a pre-trained, bundled detector that scans for brightness/contrast patterns typical of faces. BGR vs RGB -- OpenCV stores color images in Blue-Green-Red order, not the more common Red-Green-Blue; skipping conversion produces visibly wrong colors. Grayscale conversion -- face detection relies on contrast patterns, not color, so converting first is both standard and faster.

DeepFace Analysis (Deep Learning Computer Vision)

Definition:

A neural-network-based approach that goes beyond detection into higher-level facial attribute prediction -- age, gender, and emotion -- using models trained on large labeled face datasets.

Example:

DeepFace.analyze() on the same test image predicted gender (Man) and dominant emotion (Neutral, 44.71%), along with a full probability breakdown across all 7 possible emotions.

Why it matters for this internship:

The core distinction worth stating clearly: detection (OpenCV) is classical pattern matching; analysis (DeepFace) is learned, deep-learning-based understanding -- two different techniques solving two different problems.

Engineering Judgment: Pivoting to Colab

Definition:

Recognizing when a local environment is the wrong tool for the job, and switching rather than burning time fighting it.

Example:

A local pip install of TensorFlow/DeepFace was downloading at ~26 kB/s, projecting well over an hour to complete. Switching to Google Colab (pre-cached libraries, far higher bandwidth) produced the same working result in minutes.

Why it matters for this internship:

A genuinely strong interview answer: knowing when to stop fighting a slow local setup and use a better-suited tool is a real engineering decision, not a shortcut or a sign of weak setup skills.

6. Likely Interview Questions & Answers

Practice saying these out loud, in your own words, before the interview. The goal is fluency, not memorization.

Q: Why use two separate tables (PromptLog and ToolCall) instead of one?

A: They represent two genuinely different event types -- a chat exchange versus a tool invocation. Combining them would mean each row has a bunch of irrelevant empty columns depending on its type, and it would be harder to query 'all calculator calls' independently of chat history.

Q: Is your /chat endpoint connected to a real LLM?

A: Not yet -- it currently uses a rule-based fallback that echoes the prompt back, since I didn't have a live LLM API key wired in. But the architecture is built exactly as it would be with a real LLM: the endpoint takes a prompt, generates a response, and logs both to the database. Swapping in a real model just means replacing the rule-based logic with an API call -- the logging and data model don't change.

Q: Why fill missing 'age' with the median instead of the mean?

A: The median is much less affected by extreme outliers than the mean. A few unusually old or young passengers could skew the mean noticeably, while the median stays representative of a 'typical' value.

Q: Why did sex get mapped to 0/1 but embarked got one-hot encoded?

A: sex has only 2 categories with no inherent order, so a single 0/1 column is enough. embarked has 3 categories (ports) with no real order between them -- mapping them to 0, 1, 2 would falsely imply port 2 is somehow 'greater than' port 0. One-hot encoding avoids that by giving each category its own independent yes/no column.

Q: Why did Random Forest outperform a single Decision Tree?

A: A single tree can be unstable and prone to overfitting, since it learns one specific set of splits from the training data. Random Forest trains many trees on random subsets of data and features, then has them vote -- individual trees' errors tend to differ and cancel out when combined, producing a more reliable overall prediction.

Q: Why did KNN perform the worst out of your 5 models?

A: KNN predicts based on distance between data points, and we never scaled our features. fare ranges up to several hundred, while sex is just 0 or 1 -- so distance calculations were dominated by fare, drowning out the other features. Scaling the features first (e.g. with StandardScaler) would likely improve KNN specifically, since tree-based models aren't affected by feature scale the same way.

Q: What's the core difference between K-Means and the classification models you trained?

A: Classification uses labeled data -- the model learns from a known correct answer (survived or not) and is evaluated against it. K-Means is unsupervised -- it never sees the survived label at all, and just groups passengers by similarity in their raw features.

Q: What does PCA actually do, and why scale features first?

A: PCA compresses many features into fewer 'combined' axes that preserve as much of the original variation as possible, which makes high-dimensional data plottable in 2D. Scaling first matters because PCA is sensitive to feature ranges, just like KNN -- without scaling, a feature like fare would dominate the components purely because of its larger numeric range, not because it's more informative.

Q: Your PCA plot showed survivors and non-survivors mixed together, not separated. What does that mean?

A: It means those 5 features alone don't cleanly separate the two outcomes in just 2 compressed dimensions -- which is consistent with our best classifier only reaching about 81% accuracy, not near-100%. If I added sex to that feature set, I'd expect better separation, since it was one of the strongest predictors in the classification models.

Q: What's the difference between OpenCV's face detection and DeepFace's analysis?

A: OpenCV's Haar Cascade is classical, rule-based pattern matching -- it only answers 'is there a face here, and where', using brightness/contrast patterns. DeepFace uses actual deep neural networks trained on labeled face data to predict higher-level attributes like age, gender, and emotion -- a fundamentally different, learning-based approach rather than rule-based detection.

Q: Why does OpenCV need a BGR-to-RGB color conversion?

A: OpenCV historically stores color images in Blue-Green-Red order instead of the more common Red-Green-Blue order. If you skip the conversion before displaying or saving with another library, the colors come out visibly wrong -- blues and reds swapped.

Q: Why did you run the CV app on Google Colab instead of locally?

A: My local TensorFlow/DeepFace install was downloading at roughly 26 kB/s, which would have taken well over an hour. Rather than burn that much time, I switched to Google Colab, which has these libraries pre-cached and far higher bandwidth, and got the same working result in minutes. Knowing when to stop fighting a slow local environment and use a better-suited tool is a real engineering decision.